

ALU Verification Plan

Author: Bassel Mahmoud

Date: July 15, 2026

Contents

1	Author Information	1
2	Introduction	1
3	IP Design Details	1
3.1	RTL Code	1
3.2	Interface Specifications	2
3.3	Input/Output Signals	2
3.3.1	Inputs	2
3.3.2	Outputs	3
3.4	Functionality	3
3.4.1	ALU_ADD – Addition	3
3.4.2	ALU_SUB – Subtraction	3
3.4.3	ALU_AND – Bitwise AND	3
3.4.4	ALU_OR – Bitwise OR	3
3.4.5	Default / Undefined Opcode	3
3.4.6	Status Flags (Common to All Operations)	3
3.4.7	Reset Behavior	4
3.5	Latency	4
4	Verification Strategy	4
5	Constrained-Random Verification	4
5.1	Randomization Strategy	4
5.2	Constraint Classes	4
6	Sequences and Scenarios	5
6.1	Directed Sequences	5
6.2	Constrained-Random Scenario	5
7	Test Items	5
7.1	Basic Functionality	5
7.2	Arithmetic Operations	6
7.3	Logical Operations	6
7.4	Flag Verification	6

7.5	Reset Behavior	6
7.6	Corner Cases and Data Patterns	6
8	Test Case Table	7
9	Functional Coverage	8
9.1	Opcode Coverage	8
9.2	Operand Coverage	8
9.3	Flag Coverage	8
9.4	Reset Coverage	9

1 Author Information

- **IP Name:** ALU
- **Date:** 15-7-2026
- **Verifier:** Bassel Mahmoud Ahmed
- **Contact:** basselmahmoud1@gmail.com
- **Revision:** V0.1 — initial draft for the verification plan

2 Introduction

This verification plan verifies the functionality of the parameterizable Arithmetic Logic Unit (ALU) and defines the verification scope, supported operations, and the functionality of each input and output signal. The primary objective is to demonstrate and validate that the ALU correctly performs all specified arithmetic and logical operations according to the operation code (opcode). The verification process ensures the correctness of the generated result as well as the associated status flags (Zero, Negative, Carry, and Overflow). The ALU supports combinational operation with parameterizable data width and implements the following operations: addition, subtraction, bitwise AND, and bitwise OR.

3 IP Design Details

3.1 RTL Code

```
import ALU_pkg::*;
module ALU #(
    parameter WIDTH = 4
)(
    input  logic          clk, rst,
    input  wire  signed [WIDTH-1:0] a, b,
    input  alu_opcode_t   op_code,
    output logic signed [WIDTH-1:0] result,
    output logic          z, n, c, v
);
    logic signed [WIDTH-1:0] result_n;
    logic z_n, n_n, c_n, v_n;

    always_comb begin
        result_n = '0;
        c_n = 1'b0;
        v_n = 1'b0;
        case (op_code)
            ALU_ADD: begin
                {c_n, result_n} = {1'b0, a} + {1'b0, b};
                v_n = (~(a[WIDTH-1] ^ b[WIDTH-1])) & (result_n[WIDTH-1]
                    ^ a[WIDTH-1]);
            end
            ALU_SUB: begin
                // c=1 means no borrow; c=0 means borrow
                {c_n, result_n} = {1'b0, a} - {1'b0, b};
            end
        endcase
    end
endmodule
```

```

        v_n = (a[WIDTH-1] ^ b[WIDTH-1]) & (result_n[WIDTH-1] ^
        a[WIDTH-1]);
    end
    ALU_AND: result_n = a & b;
    ALU_OR : result_n = a | b;
    default: result_n = '0;
endcase
z_n = ~(|result_n);
n_n = result_n[WIDTH-1];
end

always_ff @(posedge clk) begin
    if (rst) begin
        result <= 0;
        z <= 1;
        n <= 0;
        c <= 0;
        v <= 0;
    end
    else begin
        result <= result_n;
        z <= z_n;
        n <= n_n;
        c <= c_n;
        v <= v_n;
    end
end
endmodule

```

3.2 Interface Specifications

- **Operand Width:** Parameterizable (default WIDTH = 4 bits)
- **Opcode Width:** 2 bits (encoded via `alu_opcode_t` enumeration)
- **Clock:** Single positive-edge clock
- **Reset:** Active-high synchronous reset
- **Output registration:** All outputs are registered (1-cycle latency)

3.3 Input/Output Signals

3.3.1 Inputs

Table 1: Input Signal Descriptions

Signal	Width	Description
<code>clk</code>	1	Clock input – rising-edge triggered
<code>rst</code>	1	Active-high synchronous reset
<code>a</code>	WIDTH	First signed operand
<code>b</code>	WIDTH	Second signed operand
<code>op_code</code>	2	Operation select (<code>alu_opcode_t</code> : ALU_ADD, ALU_SUB, ALU_AND, ALU_OR)

3.3.2 Outputs

Table 2: Output Signal Descriptions

Signal	Width	Description
result	WIDTH	Registered result of the selected ALU operation
z	1	Zero flag: asserted when result == 0
n	1	Negative flag: asserted when result [WIDTH-1] == 1 (MSB)
c	1	Carry/Borrow flag: set on unsigned carry-out (ADD) or borrow (SUB)
v	1	Overflow flag: asserted on signed two's-complement overflow

3.4 Functionality

3.4.1 ALU_ADD – Addition

- **result** = **a** + **b** (signed WIDTH-bit addition)
- Carry flag (**c**): unsigned carry-out of bit WIDTH
- Overflow flag (**v**): asserted when operands have same sign and result has different sign

3.4.2 ALU_SUB – Subtraction

- **result** = **a** - **b** (signed WIDTH-bit subtraction)
- Carry flag (**c**): **c**=1 means no borrow; **c**=0 means borrow occurred
- Overflow flag (**v**): asserted when operands have opposite signs and result sign differs from **a**

3.4.3 ALU_AND – Bitwise AND

- **result** = **a** & **b** (bitwise AND, WIDTH bits)
- Carry and overflow flags are cleared (0)

3.4.4 ALU_OR – Bitwise OR

- **result** = **a** | **b** (bitwise OR, WIDTH bits)
- Carry and overflow flags are cleared (0)

3.4.5 Default / Undefined Opcode

- **result_n** = 0; all flags cleared

3.4.6 Status Flags (Common to All Operations)

- Zero flag (**z**): asserted when **result_n** == 0 (combinationally computed, then registered)
- Negative flag (**n**): equals **result_n**[WIDTH-1] (sign bit), then registered

3.4.7 Reset Behavior

On `rst = 1` (synchronous): `result = 0`, `z = 1`, `n = 0`, `c = 0`, `v = 0`.

3.5 Latency

Table 3: Latency Summary

Stage	Latency
Combinational computation (<code>always_comb</code>)	0 cycles
Registered outputs (<code>result</code> , <code>z</code> , <code>n</code> , <code>c</code> , <code>v</code>)	1 clock cycle
Reset propagation	1 clock cycle

4 Verification Strategy

The ALU verification employs a layered UVM-compatible testbench using directed tests and constrained-random stimulus. The strategy targets 100% functional coverage of all opcodes, operand corner-cases, flag conditions, and reset behavior. Assertions are used to check protocol correctness on every clock edge. Coverage closure drives test generation.

5 Constrained-Random Verification

5.1 Randomization Strategy

- Operands `a` and `b` are randomized across the full signed range $[-2^{\text{WIDTH}-1}, 2^{\text{WIDTH}-1} - 1]$
- `op_code` is randomized with equal weight across all four legal opcodes
- Reset assertion is randomly inserted to verify correct flag initialization
- Targeted constraint bins are added for corner-case operand values (`0`, `±1`, `max`, `min`, `near-overflow`)

5.2 Constraint Classes

- `rand_normal`: operands uniformly distributed across full range
- `rand_corner`: operands constrained to $\{0, 1, -1, \text{MAX_VAL}, \text{MIN_VAL}, \text{MAX_VAL}-1, \text{MIN_VAL}+1\}$
- `rand_overflow`: operand pairs constrained to produce carry/overflow conditions
- `rand_zero_result`: operand pairs constrained so that `result == 0` (e.g. `a=b` for SUB, `a=b=0` for ADD)
- `rand_opcode_weighted`: opcode weighted 40% ADD/SUB, 30% AND/OR to stress arithmetic flags

6 Sequences and Scenarios

6.1 Directed Sequences

Table 4: Directed Sequences

Seq ID	Name	Description	Expected Result
SEQ_01	Reset sequence	Assert rst for 3 cycles then deassert; apply random operands	Outputs clear during reset; correct result on next cycle
SEQ_02	ADD corner	a=MAX, b=1 (overflow); a=0, b=0; a=MIN, b=-1	Correct overflow and carry flags per operation
SEQ_03	SUB corner	a=MIN, b=1 (underflow); a=0, b=0; a=MAX, b=MIN	Correct borrow/overflow flags
SEQ_04	AND all-zeros	a=0xF...F, b=0; a=0, b=0xF...F	result=0; z=1
SEQ_05	AND all-ones	a=0xF...F, b=0xF...F	result=0xF...F; n=1
SEQ_06	OR all-zeros	a=0, b=0	result=0; z=1
SEQ_07	OR all-ones	a=MAX, b=1 (WIDTH=4: 0x7 0x8)	result=0xF; n=1
SEQ_08	Zero flag check	Any op producing result=0	z=1, n=0
SEQ_09	Negative flag check	Any op producing MSB=1	n=1
SEQ_10	Opcode sweep	Cycle through all four opcodes with same operands	Correct result and flags for each opcode
SEQ_11	Back-to-back ops	100 consecutive random ops without reset	Each registered output matches reference model
SEQ_12	Reset mid-sequence	Assert rst mid-stream during active operations	Outputs revert to reset values immediately next cycle

6.2 Constrained-Random Scenario

- Run 10,000 random transactions using all constraint classes
- Interleave directed sequences with random transactions at 1:20 ratio
- Verify flags after every transaction against a SystemVerilog golden reference model

7 Test Items

7.1 Basic Functionality

- Single-cycle ADD, SUB, AND, OR operations
- All four opcodes with random operands
- Sequential operations changing opcode every cycle

7.2 Arithmetic Operations

- Addition: $a + b$ (positive, negative, mixed signs)
- Subtraction: $a - b$ (positive, negative, underflow)
- Carry generation and borrow detection
- Signed overflow detection for ADD and SUB

7.3 Logical Operations

- Bitwise AND with all-zeros, all-ones, walking patterns
- Bitwise OR with all-zeros, all-ones, walking patterns

7.4 Flag Verification

- Zero flag: asserted exactly when result is zero
- Negative flag: reflects MSB of result correctly
- Carry flag: correct for addition carry-out and subtraction borrow
- Overflow flag: correct signed overflow for ADD and SUB; cleared for AND/OR

7.5 Reset Behavior

- Reset in idle state
- Reset during active operation
- Reset release and resume normal operation

7.6 Corner Cases and Data Patterns

- Operands: $0x0$, $0x1$, -1 , $\text{MAX} (2^{\text{WIDTH}-1} - 1)$, $\text{MIN} (-2^{\text{WIDTH}-1})$
- All-zeros pattern: $a=0$, $b=0$ for all opcodes
- All-ones pattern: a and b both $0xF \dots F$
- Walking-one pattern on operand a , fixed b
- Walking-zero pattern on operand a , fixed b
- Overflow boundary: $\text{MAX}+1$, $\text{MIN}-1$ for signed arithmetic

8 Test Case Table

Table 5: Test Cases

Test ID	Test Name	Description	Expected Result
ALU_01	ADD basic	a=3, b=4, op=ADD	result=7, z=0, n=0, c=0, v=0
ALU_02	ADD carry	a=MAX, b=1 (WIDTH=4: a=7, b=1)	result=-8, c=0, v=1, n=1, z=0
ALU_03	ADD overflow pos	Two large positive operands exceeding signed range	v=1, result wraps to negative, n=1
ALU_04	ADD zero result	a=5, b=-5, op=ADD	result=0, z=1, n=0
ALU_05	SUB basic	a=5, b=3, op=SUB	result=2, z=0, n=0, c=1, v=0
ALU_06	SUB borrow	a=3, b=5, op=SUB (a < b)	result=-2, c=0 (borrow), n=1
ALU_07	SUB overflow	a=MIN, b=1, op=SUB	v=1 (signed underflow), n changes
ALU_08	SUB equal	a=b, op=SUB	result=0, z=1, c=1 (no borrow)
ALU_09	AND basic	a=0xA, b=0x6, op=AND (4-bit)	result=0x2, z=0, c=0, v=0
ALU_10	AND zero	a=0xF, b=0x0, op=AND	result=0, z=1
ALU_11	AND all-ones	a=0xF, b=0xF, op=AND (4-bit)	result=0xF, n=1
ALU_12	OR basic	a=0xA, b=0x5, op=OR (4-bit)	result=0xF, n=1, z=0
ALU_13	OR zero	a=0, b=0, op=OR	result=0, z=1
ALU_14	OR all-ones	a=0xF, b=0x0, op=OR (4-bit)	result=0xF, n=1
ALU_15	Zero flag	Any op producing result=0	z=1, n=0
ALU_16	Negative flag	Any op producing MSB=1	n=1
ALU_17	Reset idle	Assert rst=1 with no operation	result=0, z=1, n=0, c=0, v=0
ALU_18	Reset active	Assert rst=1 during ADD operation	Outputs cleared next cycle regardless of operands
ALU_19	Reset release	Deassert rst, apply ADD	Normal operation resumes; correct result next cycle
ALU_20	Walking-one a	Shift single 1-bit through a, b=0, op=OR	Result mirrors walking bit; flags correct
ALU_21	Walking-zero a	Shift single 0-bit through a (others 1), b=0xF...F, op=AND	Result has single 0 at walking position
ALU_22	All-zeros ADD	a=0, b=0, op=ADD	result=0, z=1, c=0, v=0
ALU_23	All-ones AND	a=0xF...F, b=0xF...F, op=AND	result=0xF...F, n=1, z=0
ALU_24	Opcode sweep	a=3, b=2 for all four opcodes sequentially	ADD=5, SUB=1, AND=2, OR=3; flags per operation
ALU_25	Back-to-back	50 random ops; check every registered output	Each result matches reference model

9 Functional Coverage

Functional coverage is implemented using SystemVerilog covergroups. Coverage closure (100% of all bins) is required before sign-off.

9.1 Opcode Coverage

Table 6: Opcode Covergroup

Covergroup	Coverpoint / Cross	Bins
cg_opcode	cp_opcode: op_code	ALU_ADD, ALU_SUB, ALU_AND, ALU_OR (4 bins)
cg_opcode	cp_reset_during_op: rst x op_code	Reset asserted for each opcode (4 bins)

9.2 Operand Coverage

Table 7: Operand Covergroup

Covergroup	Coverpoint / Cross	Bins
cg_operands	cp_a: operand a	zero, positive, negative, MAX, MIN (5 bins)
cg_operands	cp_b: operand b	zero, positive, negative, MAX, MIN (5 bins)
cg_operands	cp_ab_cross: a x b	(0,0), (MAX,MAX), (MIN,MIN), (MAX,MIN), (MIN,MAX) (5 cross bins)

9.3 Flag Coverage

Table 8: Flag Covergroup

Covergroup	Coverpoint / Cross	Bins
cg_flags	cp_z: zero flag	z=0, z=1 (2 bins)
cg_flags	cp_n: negative flag	n=0, n=1 (2 bins)
cg_flags	cp_c: carry flag	c=0, c=1 (2 bins)
cg_flags	cp_v: overflow flag	v=0, v=1 (2 bins)
cg_flags	cp_flag_combo: z x n x c x v	All valid flag combinations observed (16 bins)
cg_flags	cp_flags_per_op: op_code x (z,n,c,v)	Each flag state for each opcode (16 cross bins)

9.4 Reset Coverage

Table 9: Reset Covergroup

Covergroup	Coverpoint / Cross	Bins
cg_reset	cp_rst: rst transitions	de-asserted, asserted (2 bins)
cg_reset	cp_rst_duration	rst held 1 cycle, 2–5 cycles, >5 cycles (3 bins)